wso2 | Forward Deployed Engineering

POC ARCHITECTURE REVIEW

The DevOps OverSight Agent

AI-assisted incident response that correlates **Splunk** and **Datadog** over MCP to diagnose and remediate incidents in a microservice mesh — two reference architectures.

Solution 1 · Ballerina + MCP Proxy

Solution 2 · LangChain + A2A

Scott Bechtel · Internal review: FDE team + leadership · July 2026

WHAT WE'LL COVER

Agenda

01

The problem

Why cross-platform incident diagnosis is slow, risky, and manual today.

02

Level 0 architecture

The concepts both solutions share: the mesh, telemetry fan-out, MCP, correlation, the human-approval gate.

03

Solution 1 — Ballerina + MCP Proxy

One agent, one MCP endpoint that federates Splunk & Datadog behind lazy-loaded tools.

04

Solution 2 — LangChain + A2A

An orchestrator that delegates to Datadog & Splunk specialist agents over the A2A protocol.

05

Comparison & path to enterprise

An honest scorecard, the 5-minute demo, and what production hardening looks like.

THE ONE-SLIDE VERSION

Executive summary

- **The pain:** in a P1, engineers burn the first 20–40 minutes hand-correlating Splunk logs against Datadog metrics — under pressure, by hand.
- **What we built:** an AI agent that runs that cross-platform investigation automatically, then proposes a specific fix and waits for a human to approve it.
- **Two architectures:** same POC, same 5-minute demo, built two ways to show the trade-off — a Ballerina MCP Proxy and a LangChain A2A multi-agent system.
- **The durable bets:** MCP as the contract boundary, correlation kept in one reasoning context, and propose-before-act as a hard rule — never autonomous.

≤ 5 min
inject → diagnose → approve → recover

< 90s
AI diagnosis (cloud model)

0
vendor creds to demo

◆ **Bottom line**

Both approaches work. This deck gives you the scorecard to choose.

01

SECTION

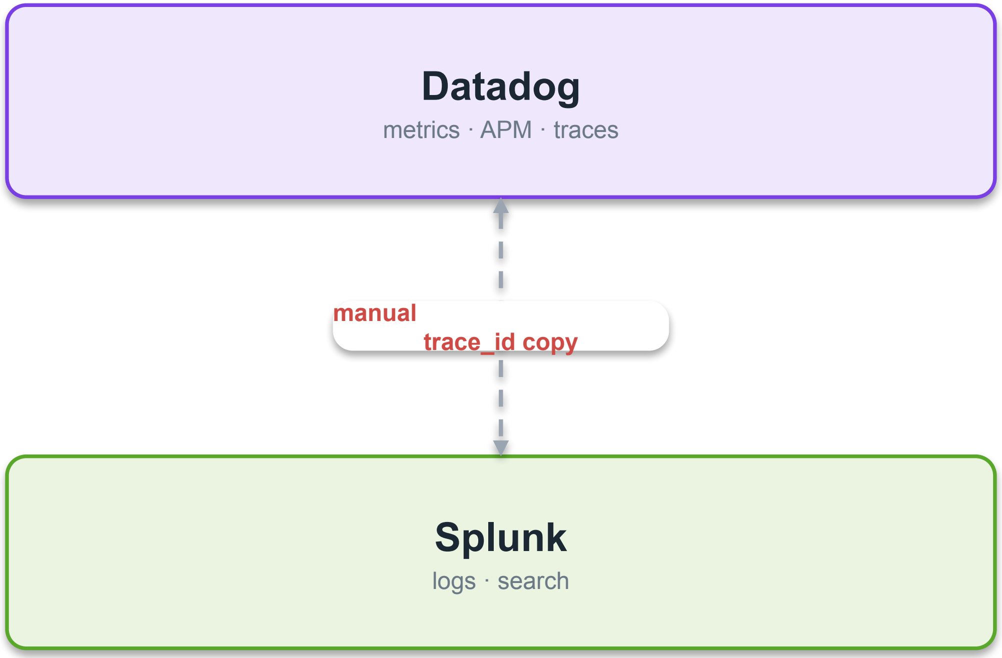
The Problem

Why cross-platform incident diagnosis is slow, risky, and manual — and what 'good' looks like.

THE PROBLEM

Best-of-breed tools, siloed by design

- **Two systems of record.** Logs and search live in Splunk. Metrics, APM, and traces live in Datadog. Both are excellent — and they don't share a query interface.
- **The join is manual.** During an incident an engineer swivel-chairs between them: spot a metric anomaly, pull a trace, copy the trace ID into a log search, cross-reference in their head.
- **It doesn't scale.** The work depends on individual institutional knowledge and gets harder as the mesh grows. Under pressure, at 2am, it's slow and error-prone.
- **Financial services raises the bar.** Any automation must be auditable, bounded, and must never change production without explicit human approval.



The engineer is the integration.

THE PROBLEM

The manual-correlation tax

1
Detect
An alert fires or a metric looks wrong in Datadog.

2
Trace
Find a representative distributed trace for a failing request.

3
Translate
Hand-convert the trace ID into a Splunk search query.

4
Cross-reference
Correlate logs, metrics, deploy history, and dependencies — mentally.

5
Hypothesize
Form a root-cause guess… or escalate to someone who knows.

20–40 min
lost per major incident

◆ **Why this is the target**

Every one of these five steps is exactly what an LLM agent with the right tools can do — fast, consistently, and with its work shown.

THE PROBLEM

Three forces shaped the design

Silo tax

Cut mean-time-to-diagnosis

Federate the two platforms so one investigation spans both — no context-switching, no manual trace-ID translation.

Approval gap

Middle path: assisted, not autonomous

Runbooks are auto-executed only after an explicit human approval. Enforced by the system, not by process or good intentions.

Lock-in risk

No AI vendor dependency

Model-agnostic by design — swap between a local model and cloud providers by config. Runs fully offline if needed.

THE PROBLEM

Who we're building for

On-call / SRE Primary user in a live incident	Plain-language root cause + blast radius + a specific action to approve — with evidence links back to both platforms.
Platform / Observability Eng Operates & extends the system	Swap model or backend by config alone; the agent's own behavior is visible in the same stack it monitors.
Security & Compliance Governance, change mgmt, audit	No state change without prior approval; a fixed action allowlist; every action recorded; secrets never in code.
Demo / Sales Engineering Presents the capability	Full incident in ~5 minutes, offline, no vendor accounts, resettable to a clean state.
Service / App owners Own individual services	Ownership, runbooks, and SLA metadata surfaced per service so response routes correctly.

THE PROBLEM

What 'good' looks like — the POC bar

< 90s
AI diagnosis (cloud)
Under 3 min on a local model

≤ 5 min
Full incident cycle
Inject → diagnose → approve → recover

2
platforms, every time
Evidence from BOTH Splunk & Datadog

0
vendor credentials
Full cycle runs offline

◆ **Propose-before-act**

The agent must list available runbooks and receive an approval signal before it ever executes one. No exceptions.

◆ **Model portability**

The same investigation must run on at least two providers (e.g. local Ollama and Anthropic) with no change beyond the provider selector.

02

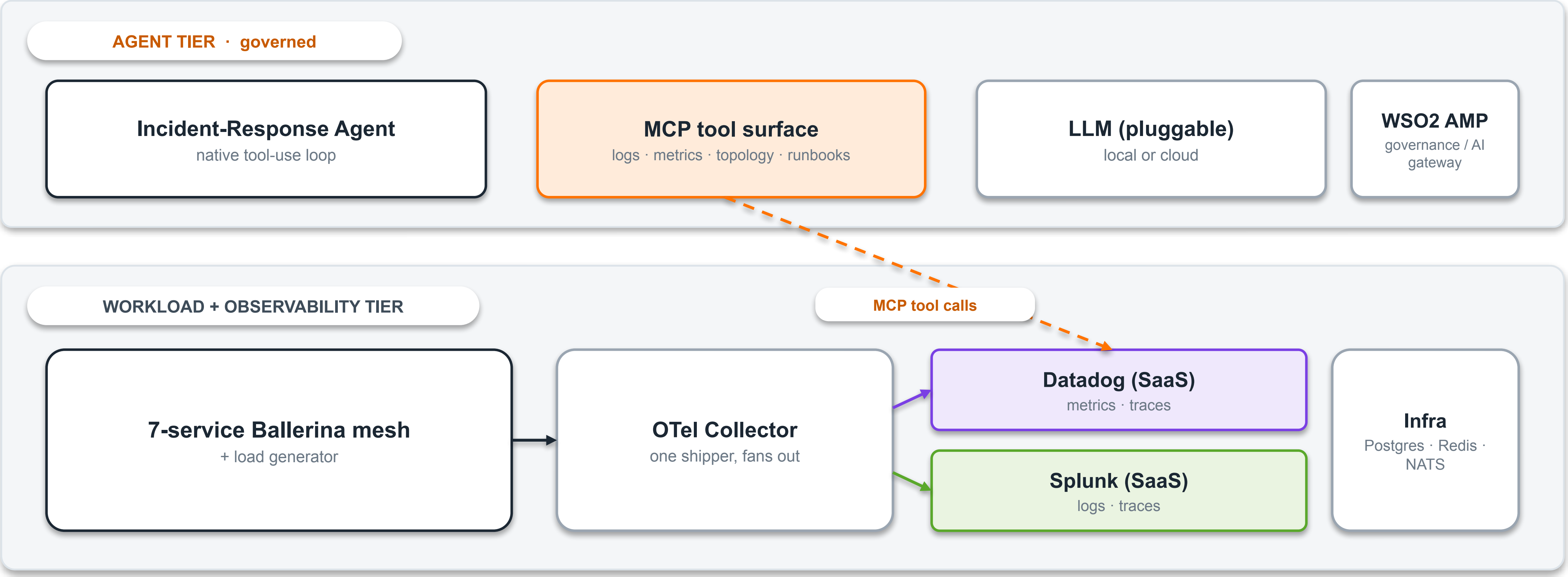
SECTION

Level 0 Architecture

The concepts both solutions share: the mesh, telemetry fan-out, MCP, correlation, and the approval gate.

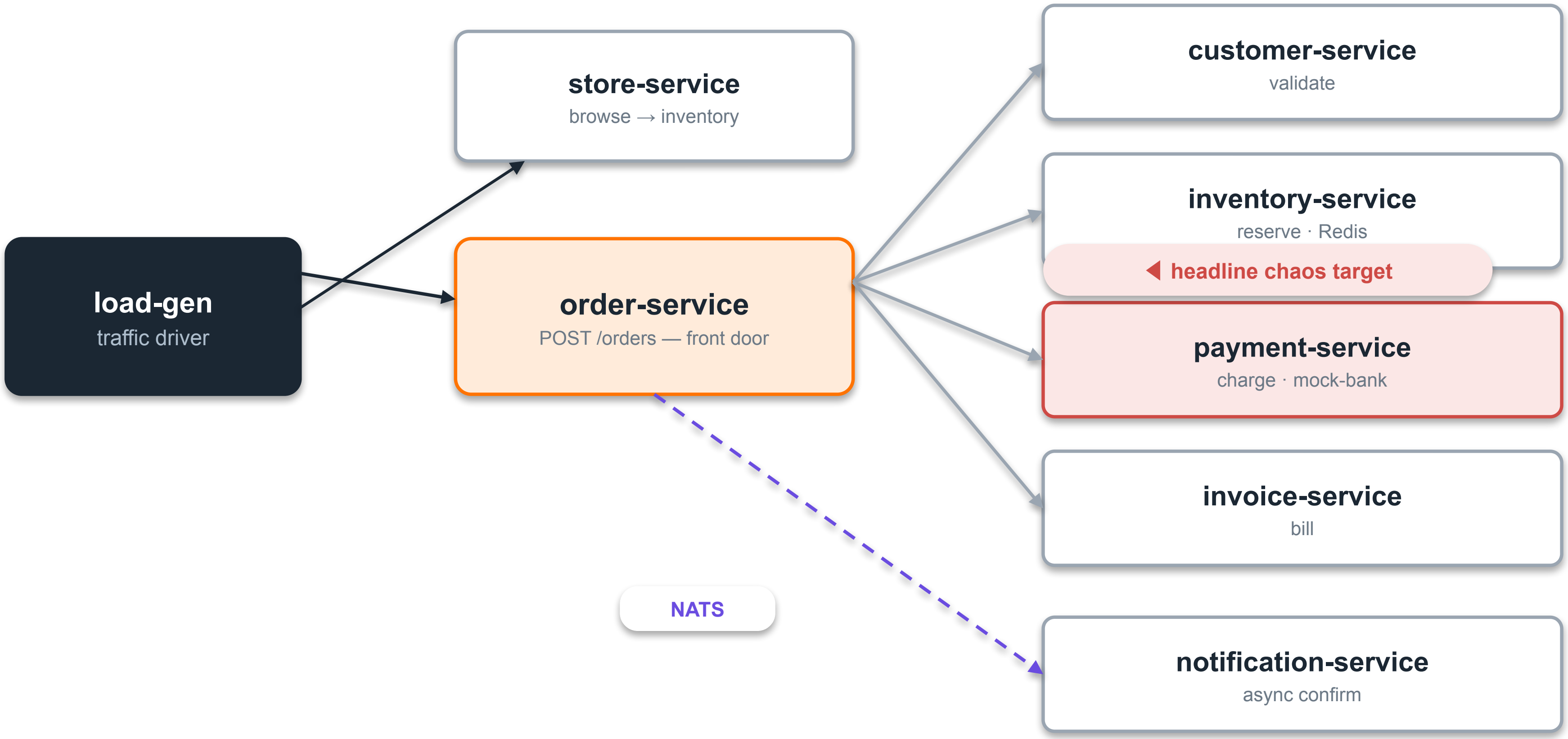
LEVEL 0 · SHARED

The big picture: two tiers



LEVEL 0 · SHARED

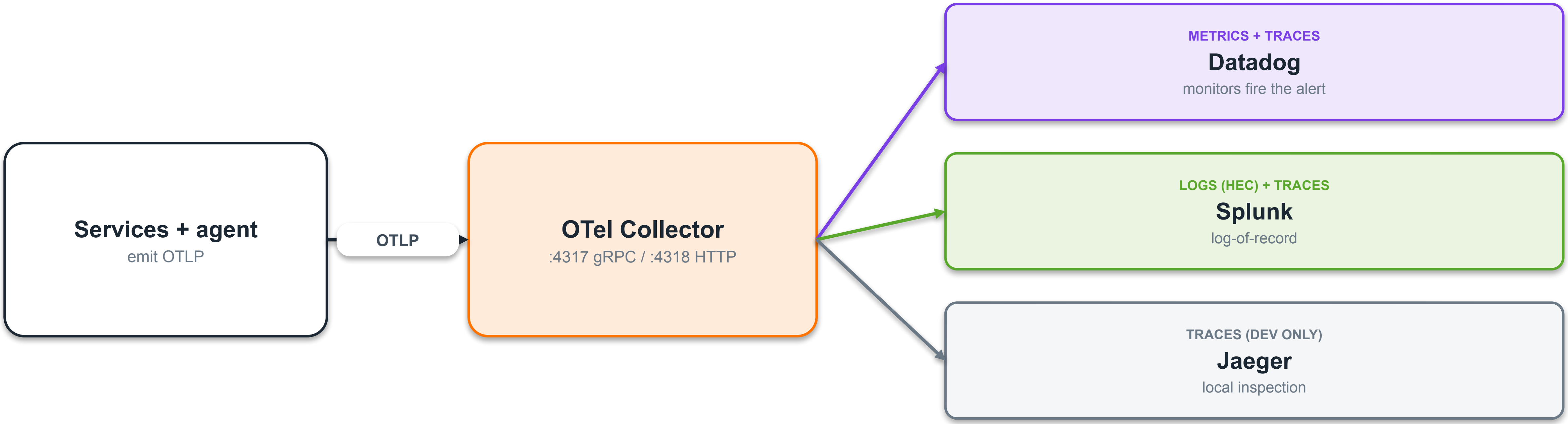
The mesh we observe & remediate



Why a mesh?

- Blast radius only exists across services.
- order fans out sync to 4 services + one async NATS hop to notification.
- Each service has a token-gated /chaos endpoint to inject faults.
- payment-service is the demo's failure point.

Telemetry fan-out: one collector, routed by signal

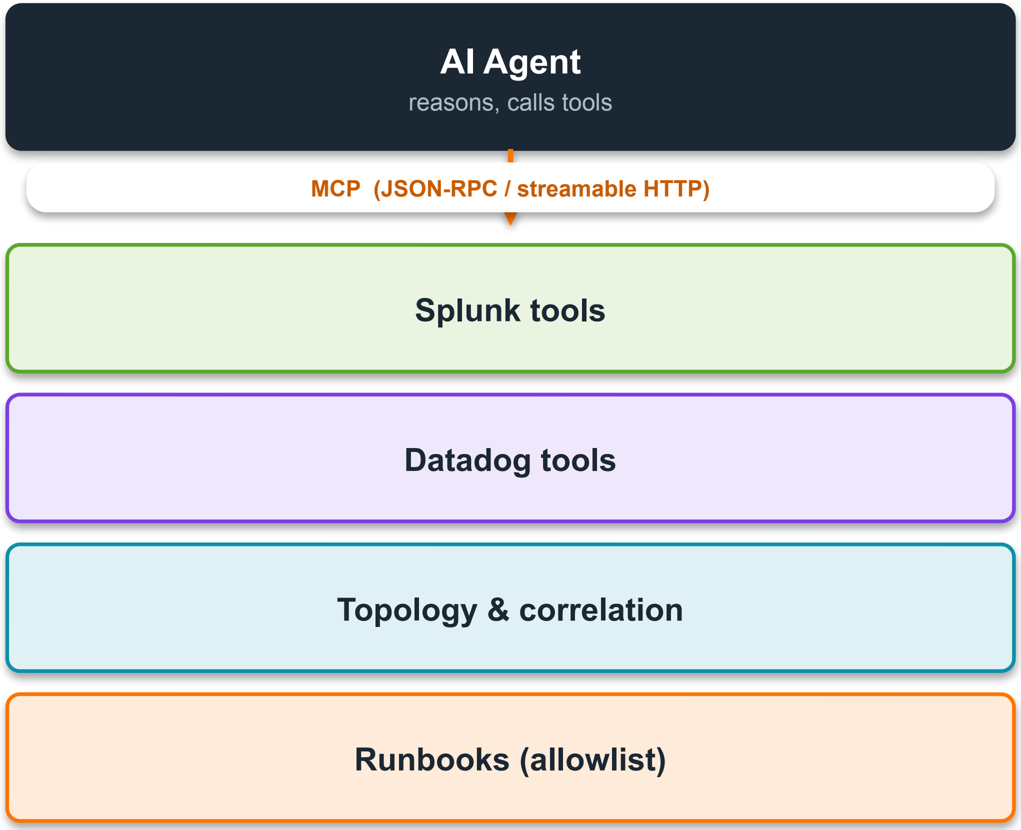


◆ **The routing rule**

Traces → both platforms (so the agent can pivot by trace_id). Logs → Splunk only. Metrics → Datadog only. DD_LOGS_ENABLED=false avoids double-billing.

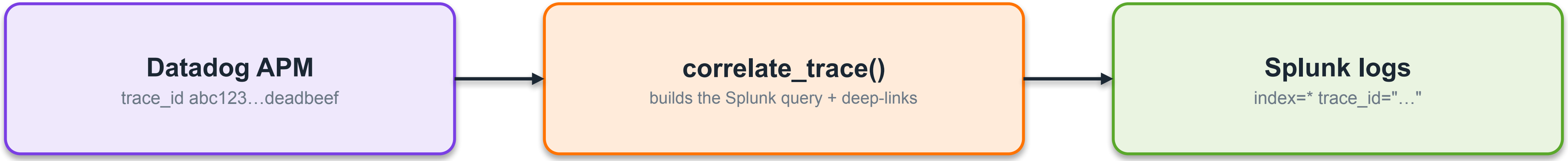
MCP: the contract boundary

- **Everything is a tool.** Splunk search, Datadog metrics, service topology, correlation, runbooks — the agent sees one uniform tool surface over MCP (Model Context Protocol).
- **Mock ⇄ live is config, not code.** Swapping a mock backend for a real vendor MCP is a URL/token change. The agent's code never changes.
- **MCP fills what vendors can't.** Splunk MCP knows logs. Datadog MCP knows metrics. Neither knows YOUR catalog, dependency graph, owners, or runbooks — we add those as local tools.
- **One place for policy.** Routing, auth, and result hygiene get a single home at the boundary.



LEVEL 0 · SHARED

The core trick: correlate by trace_id



Both platforms carry the same trace_id in structured logs — that shared key is the whole game.

◆ ⚠ The #1 correctness gotcha — trace-ID width

Datadog surfaces a 64-bit dd.trace_id; OpenTelemetry & Splunk hold a 128-bit trace_id. Search Splunk with the wrong width and the agent wrongly concludes "no logs found." The correlation layer MUST normalize both. (The mock's single demo ID masks this — real traffic exposes it.)

The agent's job — and the gate

- **Investigate.** On an incident signal, run a structured cross-platform investigation in a native tool-use loop.
- **Synthesize.** Produce a root-cause summary with blast radius and evidence links into both Splunk and Datadog.
- **Propose.** Name a specific, vetted runbook and the parameters it would use — e.g. disable-chaos on payment-service.
- **Wait.** Stop. Do nothing to production until a human approves. This is the hard line.
- **Remediate & record.** On approval, run the runbook, stream progress, write an audit entry, and post a postmortem.

Agent proposes fix

↓

🔒 **HUMAN APPROVAL GATE**
explicit approve required

↓

Runbook executes
audit logged

◆ **Bounded**

Only a fixed, pre-approved runbook list. No arbitrary command execution — ever.

End-to-end: the 10-step investigation

1. Alert / webhook triggers the agent
2. Check Datadog monitors
3. Pull metrics → find the error spike
4. Get a sample trace from APM
5. correlate_trace → build Splunk query
6. Run Splunk query → see the log evidence
7. Assess blast radius (dependencies)
8. Check recent deploys → rule in/out
9. **Propose runbook → WAIT for approval**
10. Remediate + write postmortem

One OTel trace spans the entire investigation — the agent is observable in the same stack it uses.

LEVEL 0 · SHARED

WSO2 Agent Manager's role

◆ Runtime & governance

Hosts the agent as a platform-managed workload from a Git project. Lifecycle, identity, and policy live here.

◆ AI gateway

All LLM traffic can route through AMP for rate-limiting, quota, model routing, and model-level audit — injected by config.

◆ Self-observability

The agent's own spans — reasoning, tool calls, token use, latency — flow through the same collector. Detect runaway agents.

◆ Honest scoping for tonight

In Solution 1 (Ballerina) AMP is the agent runtime. In Solution 2 (LangChain) the agents run as plain processes and AMP appears only as an optional LLM gateway. AMP is the enterprise governance plane — not a hard dependency of the POC.

No lock-in: the model is a config flag

Ollama
LOCAL · CREDS-FREE

`qwen2.5 / qwen3.5 family`

On-prem, zero cost, offline demo

Anthropic
CLOUD

`claude-sonnet-4-6`

Fastest, highest-quality tool use

OpenAI
CLOUD

`gpt-4o`

Alternative managed provider

WSO2 AMP
ENTERPRISE GATEWAY

`OpenAI-compatible`

Governed model access + audit

Switch with one variable: `LLM_PROVIDER = ollama | anthropic | openai | amp`

The default demo runs fully offline on a local model — no vendor account required.

THE FORK

Two roads to the same goal

Everything so far is shared. Here's where the two solutions diverge — and it comes down to one question:

Where do the topology/correlation tools live, and how do we keep the agent's context small?

SOLUTION 1

Ballerina + MCP Proxy

One agent talks to one MCP Proxy that federates the backends and lazy-loads their tools. Low-context by a server-side registry.

SOLUTION 2

LangChain + A2A

An orchestrator delegates to specialist agents over A2A. Low-context by decomposition — each specialist owns only its own small toolset.

03

SECTION

Ballerina + MCP Proxy

One agent, one MCP endpoint that federates Splunk & Datadog behind lazy-loaded tools.

The core idea

- **One agent, one endpoint.** The Ballerina agent connects to exactly one MCP server — the MCP Proxy on :8290. It never touches Splunk or Datadog directly.
- **The proxy federates.** Behind that one endpoint it fans out to the Splunk and Datadog MCP backends and adds local topology, correlation, and runbook tools.
- **Small context by lazy loading.** The proxy hides the big vendor tool manifests in a server-side registry and reveals only what the agent asks for.
- **Full-stack Ballerina.** Agent, proxy, mock backends, and the 7-service mesh are all Ballerina 2201.13.3 — one language, one build.

◆ **The best-practice claim**

Cross-system correlation is a single-context reasoning task. Keep it in ONE agent — federate the tools, don't fragment the reasoning.

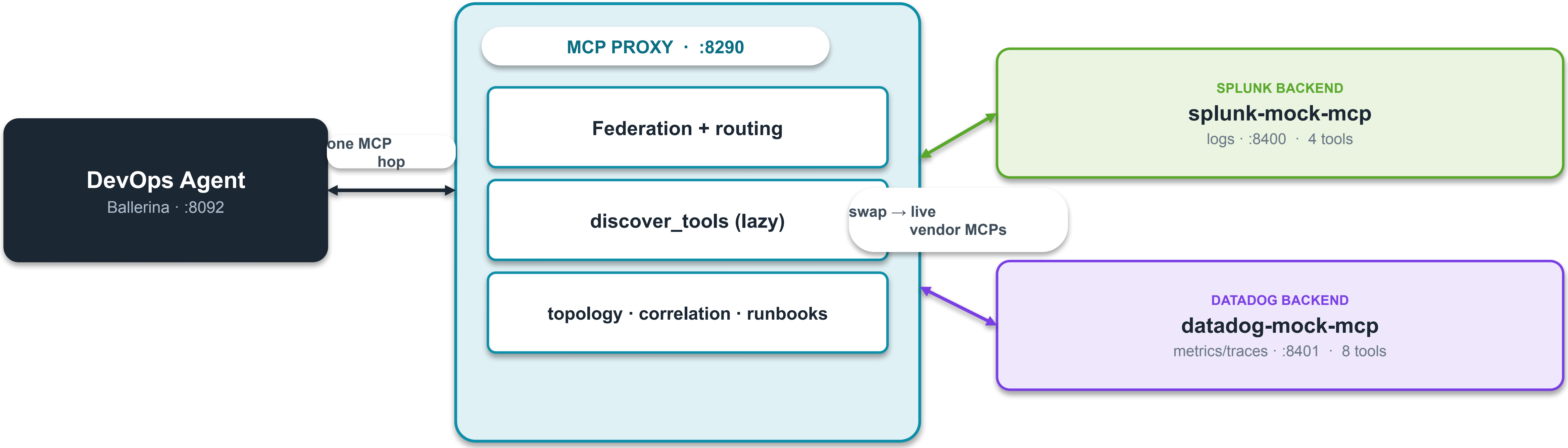
1
MCP endpoint the agent sees

12
topology tools

1
language

SOLUTION 1 · BALLERINA + MCP PROXY

Architecture



Why a proxy?

Small agent context

The vendor MCPs expose 50+ tools each. Injecting all of them would blow the context window. The proxy keeps them server-side.

Mock ⇄ live isolation

One env var on the proxy switches mock to SaaS. The agent is untouched, so you can stage a rollout safely.

Single policy chokepoint

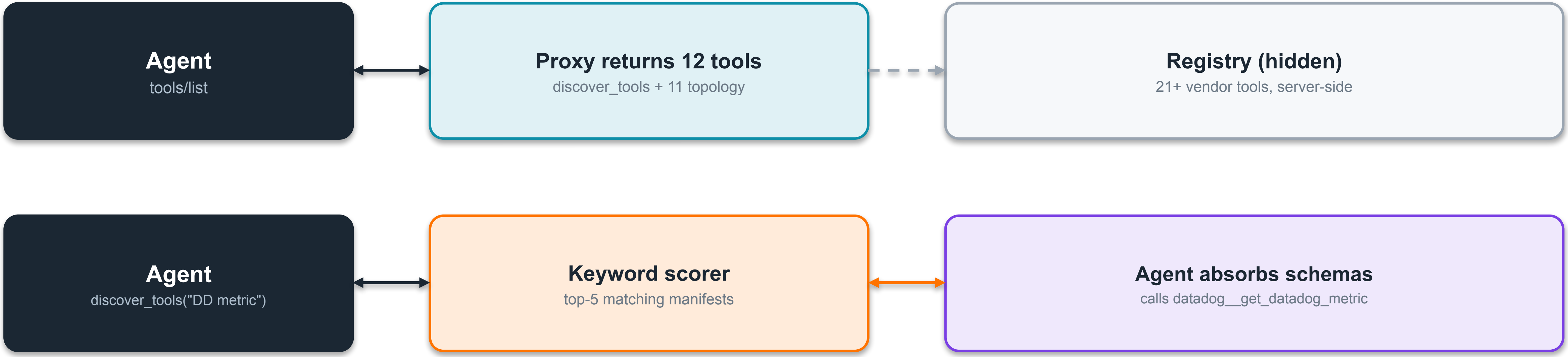
Routing, auth (via a gateway), and result hygiene get exactly one home instead of being smeared across the agent.

Fills the vendor gap

It's Ballerina, so it can also ACT — hit chaos endpoints, run runbooks — and it owns the catalog the vendors don't have.

SOLUTION 1 · BALLERINA + MCP PROXY

Lazy tool loading — the low-context pattern

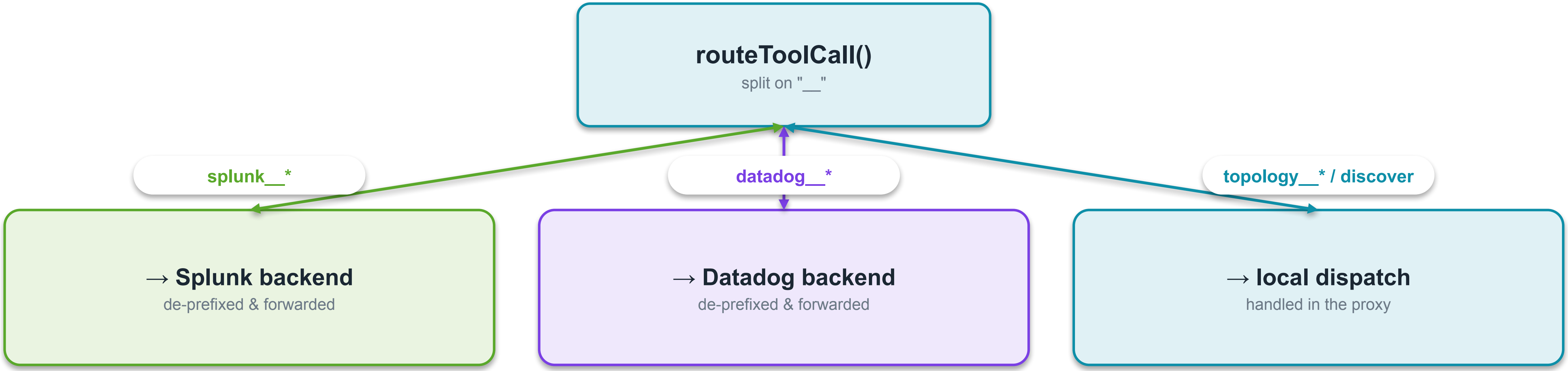


◆ **POC honesty**

Today the router is a keyword scorer — accurate enough at ~21 tools. Production swaps in a pgvector + embedding semantic router when the live MCPs bring 50+ tools each.

SOLUTION 1 · BALLERINA + MCP PROXY

Prefix routing keeps it simple



What the agent can reach: `11 topology__` (lookup / dependencies / health / correlate_trace / deploys / incidents / runbooks / audit) + `4 splunk__` + `8 datadog__`

The investigation loop in action

1. `discover_tools("Datadog metric")` → reveal the metric tools
2. `datadog__get_datadog_metric` → error-rate spike → payment-service
3. `topology__get_dependencies` → blast radius → order-service affected
4. `datadog__get_datadog_trace` → grab a failing trace
5. `topology__correlate_trace` → build the Splunk query + links
6. `splunk__splunk_run_query` → logs show mock-bank timeouts
7. `topology__find_recent_deploys` → nothing → rule out a deploy
8. `topology__list_runbooks → PROPOSE` → disable-chaos, then WAIT

Stack, tools & runbooks

Layer	Choice
Language / runtime	Ballerina Swan Lake 2201.13.3
MCP transport	Streamable HTTP · JSON-RPC 2.0
Telemetry	ballerinax/jaeger (OTLP) + prometheus
LLM loop	native tool-use, no SDK · maxTurns 30
Default model	Ollama qwen (local) / claude-sonnet-4-6

Runbooks — the fixed allowlist

- **disable-chaos** — LIVE — POST /chaos/reset. The demo's recovery lever.
- **restart-service** — stub — kubectl rollout restart
- **clear-cache** — stub — Redis FLUSHDB on inventory
- **freeze-deploys** — stub — sets an in-memory flag

Every execution appends to an in-memory audit log. No run_cli, ever.

Strengths & honest limits

✓ Strengths

- **Small, stable context** no matter how many tools the real MCPs expose.
- **Correlation in one context** logs + spans + topology join in one working set.
- **Mock ⇄ live is config** one env var, proxy-side.
- **Single policy chokepoint** routing, auth, hygiene in one place.
- **Bounded remediation** typed runbook allowlist + approval gate.
- **Clean migration path** wrap with A2A at org boundaries later.

⚠ Known limits (POC)

- **Keyword router** not vector-based; fine at ~21 tools, upgrade for 50+.
- **No result hygiene** live payloads need truncation/neutralization.
- **No endpoint auth** trusted local net; prod defers to the WSO2 gateway.
- **Static catalog** in-code map today; production reads a CMDB.
- **Stub runbooks** only disable-chaos is wired to a real action.
- **SSE buffering risk** a buffering gateway breaks streaming run_runbook.

04

SECTION

LangChain + A2A

An orchestrator that delegates to Datadog & Splunk specialist agents over the A2A protocol.

The core idea

- **Three agents.** An orchestrator (DevOpsOverSightAgent) delegates to two specialists — a DataDogAgent and a SplunkAgent — over the A2A protocol.
- **Low-context by decomposition.** Each specialist eagerly loads only its own small toolset (8 Datadog / 4 Splunk). The vendor manifests never enter the orchestrator's context.
- **No proxy, no discover_tools.** The proxy dissolves: federation becomes the A2A boundary; correlation & runbooks stay in-process in the orchestrator.
- **Familiar ecosystem.** Python 3.12, LangChain 1.x on the LangGraph runtime — the stack most teams already know.

◆ **Same principle, other shape**

A2A is used ONLY at the platform-team boundary — not to split correlation. The fusion still happens in one orchestrator context.

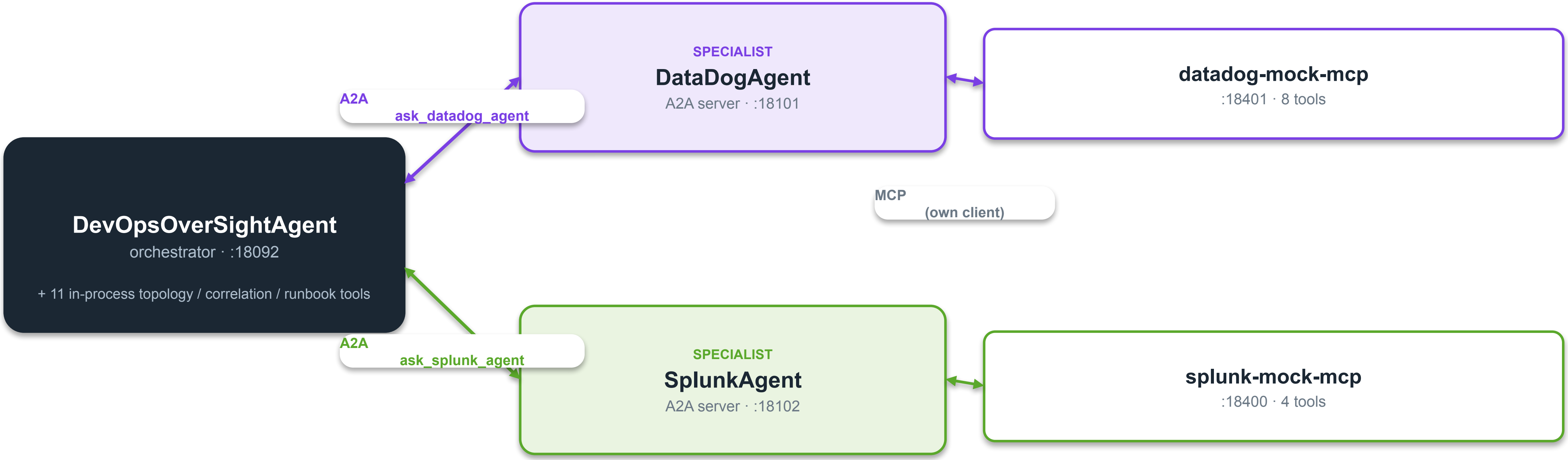
3
agents

8+4
specialist tools

Python 3.12
LangChain 1.x · LangGraph

SOLUTION 2 · LANGCHAIN + A2A

Architecture



The A2A protocol

- **Agent cards.** Each specialist publishes /.well-known/agent-card.json declaring its skill (datadog_evidence, splunk_log_search) and capabilities.
- **JSON-RPC delegation.** The orchestrator resolves the card, then sends a message and streams the reply. One-way: orchestrator → specialist.
- **Request/response, not long-running tasks.** Each call is a single round-trip — verified against the SDK, no Task lifecycle.
- **The output contract is load-bearing.** Evidence crosses as prose, so specialists MUST return trace_ids verbatim, metric values, timestamps. A vague reply starves the orchestrator.

The delegation call

orchestrator LLM

```
→ ask_datadog_agent("...")
→ A2ACardResolver.get_card()
→ SendMessageRequest (JSON-RPC)
→ specialist ReAct loop → MCP
```

◆ **Version pin matters**

a2a-sdk ≥1.1,<2 is protobuf-based. The older 0.2.x tutorials do NOT apply — build against the installed SDK.

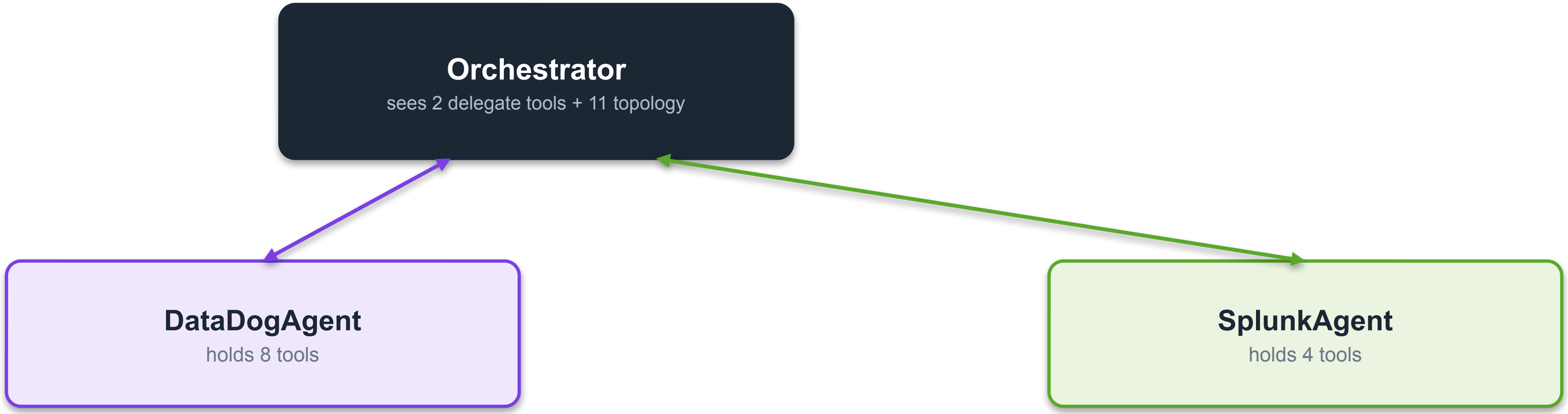
Low-context by decomposition

The problem (shared)

Real vendor MCPs expose 50+ tools each. Load them all into one agent and the context window is gone.

S2's answer

Give each platform its own agent. The orchestrator only ever sees 2 delegate tools, never the vendor manifests.



◆ **Trade**

Cleaner separation & familiar code — at the cost of another network hop and 3× the model non-determinism.

The hard approval gate

- **Code-level, not prompt-level.** A LangGraph HumanInTheLoopMiddleware physically interrupts the graph before topology__run_runbook executes.
- **The graph pauses.** /investigate returns the proposal + a sessionId. State is checkpointed by InMemorySaver keyed to that session.
- **Human resumes it.** Operator POSTs "approve" to /chat; the graph resumes with a Command(resume=...) and only then runs the runbook.
- **Fail-safe by default.** Any non-approval reply is treated as rejection. The runbook does not run.

LLM calls run_runbook

↓

⛔ **GRAPH INTERRUPT**

execution physically paused

↓ **human POSTs "approve"**

Graph resumes → runbook runs

◆ **Why it's stronger**

The model literally cannot execute without a recorded approval — it's enforced by the runtime.

The investigation loop with A2A

1. `ask_datadog_agent(...)` → A2A → DataDogAgent → monitors, metrics, sample trace_id
2. `topology__correlate_trace(id)` → in-process → normalize width, build SPL + links
3. `ask_splunk_agent(SPL)` → A2A → SplunkAgent → 502 logs, no deploys
4. `topology__get_dependencies` → in-process → blast radius
5. `topology__find_recent_deploys` → in-process → none → chaos suspected
6. `run_runbook → INTERRUPT` → graph pauses → returns proposal + sessionId
7. `POST /chat "approve"` → graph resumes → disable-chaos → recovery

A2A delegation · in-process tool · gate

Stack & the Timeout Chain

Layer	Choice
Language / runtime	Python 3.12 · single uv workspace
Agent framework	LangChain 1.x on LangGraph 1.x (create_agent)
A2A	a2a-sdk ≥1.1,<2 (protobuf)
MCP	official mcp SDK · langchain-mcp-adapters
Gate / state	HumanInTheLoopMiddleware + InMemorySaver

The Timeout Chain (must stay ordered)

uvicorn `600s`

A2A client `300s`

sub-agent LLM `180s`

MCP call `30s`

Largest-outermost. assert_timeout_chain() fails fast at startup if misordered.

Strengths & honest limits

✓ Strengths

- **Familiar ecosystem** Python + LangChain; low ramp for most teams.
- **Hard, code-level gate** graph interrupt, not a prompt instruction.
- **Structural low-context** specialists cap tools by construction.
- **Clean protocol separation** A2A at the boundary, MCP for tools, in-process fusion.
- **Same model flexibility** 4 providers, config-swappable.
- **Full self-observability** one trace spans user → A2A → specialist → MCP.

⚠ Known limits (POC)

- **3× model non-determinism** three agents must each emit clean tool calls.
- **Prose output contract** vague specialist replies starve correlation.
- **Extra network hop** A2A adds latency vs an in-process call.
- **Timeout Chain to maintain** four nested timeouts, order-sensitive.
- **In-memory state** restart mid-approval drops the pending runbook.
- **SDK version drift** a2a-sdk 1.1 is protobuf; pin carefully.

05

SECTION

Comparison & Path Forward

An honest scorecard, the 5-minute demo, and what production hardening looks like.

COMPARISON

The architectural crux

Both keep correlation in ONE reasoning context. They differ only in how they keep the toolset small.

Solution 1 — federate the tools

- **Low-context via** a proxy with a server-side registry + lazy discover_tools.
- **Correlation locus** one Ballerina agent holds all evidence directly.
- **Boundary** one process federates; nothing is fragmented.

Solution 2 — decompose the agents

- **Low-context via** specialist agents that each own a small toolset.
- **Correlation locus** the orchestrator fuses prose evidence from specialists.
- **Boundary** A2A at the platform-team line — NOT inside correlation.

COMPARISON

Side-by-side scorecard

Dimension	Solution 1 · Ballerina + Proxy	Solution 2 · LangChain + A2A
Low-context strategy	Proxy lazy-load (server registry)	Agent decomposition (specialists)
Correlation locus	One agent, direct	Orchestrator fuses prose
Approval gate	Prompt-level instruction	Code-level graph interrupt
Language / stack	Ballerina 2201.13.3	Python 3.12 / LangChain 1.x
Moving parts	Fewer — 1 agent + proxy	More — 3 agents + hop
Model non-determinism	×1 (single agent)	×3 (each specialist)
Ecosystem familiarity	Ballerina — niche	Python/LangChain — broad
Tool-sprawl at 50+ tools	Needs vector router upgrade	Handled by decomposition
WSO2 AMP fit	Native agent runtime	Optional LLM gateway only
Ops surface	Single reasoning process	Distributed; timeout chain

Recommendation — measured on two models

LOCAL — qwen2.5:14b (Ollama)

18 runs each	LangChain	Ballerina
Correct-proposal	61% (11/18)	28% (5/18)
Median latency	62 s	78 s
LLM calls / inv.	~19	~9

CLOUD — Claude Haiku 4.5

18 runs each	LangChain	Ballerina
Correct-proposal	100% (18/18)	100% (18/18)
Median latency	36 s	14 s
LLM calls / inv.	~19	~9

What the two models say

- **Model choice dominates reliability.** Local 28–61% → cloud 100% for both. The 14B's flakiness (it narrates a tool call instead of making it, and quits early) is a model problem, not an architecture one — don't ship the local model.
- **The speed winner FLIPS with the model.** Slow local → LangChain wins (small, cheap calls). Fast cloud → Ballerina wins ~2.6× (14 vs 36 s): its ~9 calls beat LangChain's ~19 network round-trips. 'Fewer round-trips = faster' holds — but only once inference is fast.
- **On the model you'd actually ship (cloud):** both 100% reliable, Ballerina meaningfully faster. So runtime no longer decides it — the qualitative scorecard does, on client context: Ballerina = fewer parts + WSO2-native; LangChain = code-level gate + Python familiarity.

Method: identical model per column; 3 datasets × 6 runs each (warm-up discarded); stacks run sequentially, never concurrent; mock MCP backends; same chaos + payload. Local = qwen2.5:14b on Ollama; cloud = claude-haiku-4-5 via Anthropic. Latency = time-to-proposal via curl.

COMPARISON

The 5-minute demo — payment-service P1

0:00

Inject chaos

payment-service starts returning 502s + 2s latency.

0:45

Signals diverge

Datadog monitor fires; Splunk logs fill with errors.

1:15

Investigate

Agent runs the cross-platform loop, correlates by trace_id.

3:30

Propose & approve

Agent proposes disable-chaos; operator approves.

4:15

Recover

Runbook runs; mesh recovers; agent posts a postmortem.

◆ **Runs offline, resets clean**

No vendor credentials. A token-gated chaos endpoint injects the fault; disable-chaos (or reset) returns to a clean baseline for the next run.

COMPARISON

POC → Enterprise: the hardening path

Capability	POC state	Enterprise requirement
Identity & access	Unauthenticated local net	OIDC/SSO + workload identity + per-tool RBAC at the gateway
Secrets	.env variables	Vault / cloud secrets manager; rotation
Audit log	In-memory, session-scoped	Durable, tamper-evident, tied to identity, SIEM-fed
Approval gate	S1 prompt / S2 code-level	Hard code-level interception in both; recorded signal
Service catalog	Static in-code map	CMDB-backed, team-maintained, versioned
Observability backends	Mock MCP servers	Live Splunk (app 7931) + Datadog (Bits AI) MCPs
Trace-ID handling	Verbatim (demo id)	True 64/128-bit normalization on real traffic
Scale	Single node	Agents on K8s; proxy behind LB; managed infra

COMPARISON

Cross-cutting gotchas to respect

Trace-ID width (64 vs 128-bit)

The #1 correctness bug. Normalize or the agent sees 'no logs.' Mocks mask it.

NATS async propagation

HTTP trace context auto-propagates; NATS does not. Inject traceparent into the envelope.

Untraced Postgres

Turn on SQL connector tracing or DB latency is invisible to the agent.

SSE buffering at the gateway

A buffering proxy breaks streaming run_runbook progress.

Model non-determinism (Ollama)

Keep maxTurns ≥ 25; prefer a cloud model for the live demo.

In-memory state

Audit log & pending approvals are volatile — a restart drops them.

WRAP-UP

Summary & next steps

What we proved

- **The capability works.** cross-platform diagnosis + approved remediation in a 5-minute, offline, laptop demo.
- **Two credible architectures** — Ballerina + MCP Proxy and LangChain + A2A — each with clear trade-offs.
- **The durable bets hold:** MCP as the contract, single-context correlation, propose-before-act, model portability.
- **A clean path to production** exists for either — a hardening checklist, not a re-architecture.

NEXT STEPS

1 Fill in the recommendation with the client's team & governance context.

2 Pick one architecture for a hardening spike (auth + live MCPs).

3 Wire real Splunk & Datadog MCPs; exercise trace-ID normalization.

4 Run the demo live with the client on their own laptop.

Ports & endpoints reference

Solution 1 · Ballerina	Port
DevOps Agent	8092 → 8000
MCP Proxy	8290
splunk-mock-mcp	8400
datadog-mock-mcp	8401
payment chaos endpoint	9196
Prometheus scrape	9797

Solution 2 · LangChain	Port
Orchestrator	18092 → 8000
DataDogAgent (A2A)	18101
SplunkAgent (A2A)	18102
datadog-mock-mcp	18401
splunk-mock-mcp	18400
OTLP collector	14317 / 14318

Shared: demo trace_id `abc123def4567890123456789deadbeef` · WSO2 AMP console `:3000` · chaos token `dev-chaos-token`

The LangChain stack 1-prefixes every host port so both stacks run side-by-side for a live A/B on one machine.

wso2

Thank you.

Questions, pushback, and better ideas all welcome — this is a team effort.

Scott Bechtel · WSO2 Forward Deployed Engineering